UVSQ université PARIS-SACLAY | UFR des Sciences CAMPUS DE VERSAILLES

M1 Informatique

Méthodes de Ranking

Comparaison expérimentale de variantes du PageRank

Réalisé par :

Manal BOUZEGGAOUI

Cyrus NOUKPOZOUNKOU

Année 2025–2026

Abstract

Ce rapport présente une étude comparative de trois variantes de l'algorithme PageRank, implémentées en C++ et évaluées sur trois graphes web de tailles croissantes : Harvard500 (500 nœuds), web-Google (875 000 nœuds) et uk-2002 (18,5 millions de nœuds). La première variante est la méthode des puissances classique. Les deux suivantes reposent sur un algorithme d'encadrement qui maintient deux vecteurs $X^{(k)}$ et $Y^{(k)}$ encadrant la distribution stationnaire x à chaque itération, avec deux organisations mémoire différentes : vecteurs séparés et vecteurs entrelacés. L'étude porte sur l'influence des paramètres α (facteur d'amortissement) et ε (critère de convergence) sur le nombre d'itérations, le temps d'exécution et la qualité des résultats. Les expériences révèlent que l'encadrement est plus coûteux en itérations mais produit des scores plus précis pour un même ε, et que l'entrelacement mémoire apporte un gain significatif sur les graphes de grande taille.

Contents

Introduction **4**

1 Fondements théoriques **5**
1.1 Modélisation du web comme chaîne de Markov 5
1.2 La matrice Google G 5
1.3 Le facteur d'amortissement α 6
1.4 Les vecteurs ∇ et Δ 6

2 Choix de conception **7**
2.1 Représentation par listes chaînées 7
2.2 Représentation CSC 7
2.3 Gestion des nœuds puits (Dangling Nodes) 8
2.4 Calcul de la norme 9
2.5 Parallélisation 9

3 Algorithme d'encadrement **10**
3.1 Présentation 10
3.2 Remarque sur l'invariant d'encadrement 10

4 Structures de données pour l'encadrement **11**
4.1 Structure 1 : nabla/delta distincts et quatre vecteurs X/Y . . 11
4.2 Structure 2 : nabla/delta entrelacés et deux vecteurs X/Y entrelacés 11

5 Environnement expérimental **13**

6 Méthodologie d'évaluation **14**
6.1 Démarche générale 14
6.2 Raison d'arrêt 14
6.3 Mesure de la qualité 14
6.4 Référence de qualité 14
6.5 Reproductibilité 15

7 Résultats et analyse **16**
7.1 Influence du facteur d'amortissement α 16
7.2 Impact de l'entrelacement mémoire 18
7.3 Influence du critère de convergence ε 19
7.4 Qualité des résultats en fonction de ε 19
7.5 Accord sur le Top 20 20

8 Structure retenue **22**
8.1 Justification 22

9 Difficultés rencontrées et perspectives **23**
9.1 Invariant d'encadrement et normalisation séparée 23
9.2 Conséquences et ce qu'on aurait pu obtenir 23
9.3 Autres difficultés 24

10 Conclusion **25**

Introduction

Le PageRank, introduit par Brin et Page en 1998, est l'algorithme fondateur du moteur de recherche Google. Il modélise le comportement d'un internaute naviguant aléatoirement sur le web et attribue à chaque page une importance proportionnelle à la probabilité qu'il s'y trouve après un grand nombre de clics. Mathématiquement, le PageRank correspond à la distribution stationnaire d'une chaîne de Markov dont la matrice de transition est construite à partir du graphe des hyperliens.

Ce rapport a pour objectif de comparer expérimentalement trois implémentations de cet algorithme, en étudiant l'influence des paramètres fondamentaux α et ε sur leurs performances respectives. La première implémentation est la méthode des puissances classique, qui constitue la référence. Les deux suivantes reposent sur un algorithme d'encadrement proposé dans l'énoncé du projet, qui construit à chaque itération deux vecteurs $X^{(k)}$ et $Y^{(k)}$ encadrant la distribution stationnaire exacte, avec deux organisations mémoire différentes visant à optimiser la localité cache.

Le rapport est organisé comme suit. La section 1 présente les fondements théoriques du PageRank et la construction de la matrice Google. La section 2 détaille les choix de conception : stockage CSC, gestion des nœuds puits, organisations mémoire candidates pour les vecteurs X et Y, parallélisation OpenMP et fusion du calcul de norme. La section 3 décrit l'algorithme d'encadrement. La section 4 présente les deux structures de données envisagées pour stocker ∇ et Δ. La section 5 présente l'environnement expérimental et les matrices utilisées. La section 6 expose la méthodologie d'évaluation. La section 7 présente et interprète les résultats expérimentaux. La section 8 présente la structure retenue et sa justification à la lumière des résultats. La section 9 discute des difficultés rencontrées et perspectives. La section 10 conclut le rapport.

1 Fondements théoriques

1.1 Modélisation du web comme chaîne de Markov

Le web est modélisé comme un graphe orienté $G = (V, E)$ où V est l'ensemble des pages et E l'ensemble des hyperliens. On note $n = |V|$ le nombre de pages. À partir de ce graphe, on construit une matrice de transition P de taille $n \times n$ où :

$$P[i, j] = \frac{1}{d_i} \text{ si } (i, j) \in E, \quad 0 \text{ sinon}$$

avec d_i le degré sortant de la page i. Chaque ligne de P représente la distribution de probabilité de transition depuis la page i.

1.2 La matrice Google G

La matrice de transition P peut contenir des **nœuds puits**, c'est-à-dire des pages sans lien sortant. Ces nœuds correspondent à des lignes entièrement nulles dans P, ce qui brise la propriété stochastique de la matrice et empêche la convergence vers une distribution stationnaire unique.

Pour corriger cela, on introduit la **matrice Google** G définie par :

$$G = \alpha P + \alpha \frac{1}{N} f e^t + (1 - \alpha) \frac{1}{N} e e^t$$

où :

- P est la matrice de transition du graphe web
- $\alpha \in (0, 1)$ est le facteur d'amortissement
- f est le vecteur indicateur des nœuds puits : $f(i) = 1$ si $\deg_{\text{sortant}}(i) = 0$, 0 sinon
- e est le vecteur de taille N ne contenant que des 1

Le terme $\alpha \frac{1}{N} f e^t$ redistribue uniformément la probabilité depuis les nœuds puits, tandis que $(1-\alpha) \frac{1}{N} e e^t$ modélise une téléportation uniforme vers n'importe quelle page avec probabilité $1 - \alpha$. G est ainsi une matrice stochastique, irréductible et apériodique, ce qui garantit l'existence et l'unicité de la distribution stationnaire x vérifiant :

$$x = xG$$

1.3 Le facteur d'amortissement α

Le facteur d'amortissement $\alpha \in (0, 1)$ contrôle l'équilibre entre deux comportements de l'internaute :

- Avec probabilité α, il suit un hyperlien de la page courante
- Avec probabilité $1 - \alpha$, il se téléporte aléatoirement vers une page quelconque du web

1.4 Les vecteurs ∇ et Δ

$$\nabla[j] = \min_i G[i, j] \qquad \Delta[j] = \max_i G[i, j]$$

Ces vecteurs représentent respectivement le minimum et le maximum de chaque colonne de G. Ils servent de base à la construction d'un encadrement de la distribution stationnaire x à chaque itération de l'algorithme. On a en effet la propriété suivante :

$$\forall k, \forall i, \quad X^{(k)}[i] \leq x[i] \leq Y^{(k)}[i]$$

où $X^{(k)}$ et $Y^{(k)}$ sont deux vecteurs construits par itération à partir de ∇ et Δ, et x est la distribution stationnaire exacte. Cet encadrement garantit une borne sur l'erreur d'approximation à tout moment de l'algorithme.

2 Choix de conception

2.1 Représentation par listes chaînées

La première structure de données utilisée pour représenter la matrice P est un tableau avec des listes chaînées. On utilise un tableau de N où chaque élément pointe sur une liste chaînée. Pour chacune de ces listes chaînées, chaque nœud stocke uniquement les entrées non nulles de la colonne correspondante.

Formellement, on définit deux structures :

```
struct Element {
    int row_index;
    double value;
    struct Element* next;
};

struct Matrice {
    int n;
    long long n_links;
    Element** columns;
};
```

`Matrice` contient la taille N du graphe, le nombre total de liens `n_links`, et un tableau de pointeurs `columns` de taille N. Pour une colonne j donnée, `columns[j]` pointe vers la liste chaînée contenant exactement les paires `(row_index, value)` pour lesquelles $P[i, j] \neq 0$. Les entrées nulles ne sont pas stockées, ce qui réduit significativement l'occupation mémoire. Cependant, cette réprésentation présentent certains défaut comme :

- un surcoût dû aux pointeurs `next` : chaque nœud stocke en plus de `row_index` et `value` un pointeur de 8 octets,
- des accès non contigus en mémoire (saut de mémoire), ce qui provoque des **cache misses** fréquents et implique un temps de calcul long.

2.2 Représentation CSC

Dans le but d'améliorer la manière de stocker la matrice, nos recherche nous on conduit au format CSC. Le format CSC (Compressed Sparse Column) stocke la matrice creuse à l'aide de trois tableaux compacts plutôt qu'une structure chaînée.

Formellement :

```
struct Matrice{
    int n;
    long long n_links;
    double* values;
    long long* col_ptr;
    int* row_idx;
    int* degree_Sort;
};
```

- `n` : taille du graphe N
- `n_links` : nombre total de liens
- `values` : tableau de taille `n_links` contenant les valeurs non nulles de la matrice, colonne par colonne
- `col_ptr` : tableau de taille $N + 1$ où `col_ptr[j]` donne l'indice de début des entrées de la colonne j dans `values` et `row_idx`
- `row_idx` : tableau de taille `n_links` contenant les indices de ligne associés à chaque valeur. Les destinations d'un même sommet source se suivent dans le tableau
- `degree_Sort` : tableau de taille N contenant le degré sortant de chaque nœud

Ce format est plus compact (pas de pointeurs par émément, seulement trois tableaux contigus), ce qui est favorable au **prefetching** du processeur et conduit à un gain de mémoire, temps de chargement et de temps de calcul. Pour ces raisons, nous avons retenu le **format CSC** pour l'implémentation.

Mais il faut noter que s'il n'y a pas assez de mémoire, il faut lire deux fois le fichier pour construire la matrice. La première lecture, permet la création des différents vecteurs et permet de connaître le degré sortant pour chaque sommet. Cette information est cruciale pour savoir à quel indice il faut commencer par insérer les destinations d'un sommet lors de la deuxième lecture. En cas de mémoire suffisante, on peut garder les données en mémoire et effectuer deux boucles pour construire la matrice.

2.3 Gestion des nœuds puits (Dangling Nodes)

Pour la gestion des puits, au lieu de construire le vecteur f , on a fait le choix d'exploiter directement le tableau `degre_sort` issu de la phase de construction CSC. Ainsi, pour calculer le scalaire xf, nous utilisons un

`if-statement` dans la boucle de calcul. Ce choix se justifie par le fait que la donnée `degre_sort` est déjà sollicitée pour le calcul de $P[i,j] = \frac{1}{d_i}$ ou $P[i,j] = \frac{poids}{d_i}$ lors des itérations.

Après, on s'est dit qu'on pourrait tout à fait constuire un vecteur f de taille Z(nombre de sommets ayant un dégré sortant nul) qui contiendrait l'indice de ces sommets. Après l'avoir l'implémenter, il s'est avéré moins efficace que notre première approche. Cela est dû à l'accès indirect que fait le processeur pour connaitre les indices d'itérations lors de notre calcul de xf.

2.4 Calcul de la norme

Dans la méthode classique, le calcul de la norme est souvent traité comme une étape post-itérative distincte, nécessitant un balayage complet des vecteurs en mémoire. Pour notre simulateur, nous avons implémenté une **fusion de réduction** (*fused reduction*).

Mécanisme de fusion : Au lieu d'attendre la fin de l'itération pour évaluer la convergence, la norme L_1 de l'écart est accumulée **simultanément** au calcul de la nouvelle valeur de x_i. Concrètement, dès que la composante $x_i^{(k+1)}$ est générée dans le registre du processeur, nous calculons immédiatement sa contribution à la norme totale : $\delta = \delta + |x_i^{(k+1)} - x_i^{(k)}|$. Ainsi, à la fin du calcul de la nouvelle distribution, on obtient déjà la norme.

Nous avons maintenu cette même logique pour calculer la norme avec nos vecteurs ∇ et Δ. Donc $\delta = \delta + |y_i - x_i|$ pour la première organisation de X et Y, et $\delta = \delta + |z_i - z_{i+1}|$ pour la seconde organisation.

2.5 Parallélisation

Les boucles internes de calcul du produit vecteur-matrice sont parallélisées avec OpenMP via des directives `#pragma omp parallel for` et des réductions sur les normes. Les expériences elles-mêmes sont exécutées séquentiellement afin de préserver la validité des mesures de temps.

3 Algorithme d'encadrement

3.1 Présentation

L'algorithme d'encadrement construit par itération deux vecteurs $X^{(k)}$ et $Y^{(k)}$ tels que :

$$\forall k,\ \forall i, \quad X^{(k)}[i] \leq x[i] \leq Y^{(k)}[i]$$

où x est la distribution stationnaire exacte. Il est initialisé et mis à jour comme suit :

1. $X^{(0)} \leftarrow \dfrac{\nabla}{\|\nabla\|_1}$
2. $Y^{(0)} \leftarrow \dfrac{\Delta}{\|\Delta\|_1}$
3. À chaque itération k :
   (a) $X \leftarrow \max\big(X^{(k)}G,\ \nabla\big)$, puis $X^{(k+1)} \leftarrow \dfrac{X}{\|X\|_1}$
   (b) $Y \leftarrow \min\big(Y^{(k)}G,\ \Delta\big)$, puis $Y^{(k+1)} \leftarrow \dfrac{Y}{\|Y\|_1}$
   (c) Jusqu'à ce que $\|X^{(k+1)} - Y^{(k+1)}\|_1 < \varepsilon$

À convergence, le score PageRank de chaque page i est estimé par la moyenne $\frac{X_i+Y_i}{2}$.

3.2 Remarque sur l'invariant d'encadrement

D'un point de vue théorique, l'invariant $X^{(k)}[i] \leq x[i] \leq Y^{(k)}[i]$ est garanti à chaque itération. En pratique, la normalisation séparée de X et Y — chacun divisé par sa propre norme L_1 — peut introduire des inversions $X[i] > Y[i]$ pour certaines composantes. En effet, si $\|\nabla\|_1 \ll \|\Delta\|_1$, la division par des normes très différentes peut perturber l'ordre composante par composante. Nous avons observé expérimentalement que ces inversions persistent tout au long des itérations, mais que leur amplitude à convergence est de l'ordre de 5×10^{-9}, ce qui est négligeable devant tout ε raisonnable. L'algorithme converge correctement malgré cet artefact numérique.

4 Structures de données pour l'encadrement

L'algorithme d'encadrement nécessite de stocker les vecteurs ∇ et Δ en plus des vecteurs X et Y. L'organisation mémoire de ces quatre éléments influe directement sur les performances. Deux combinaisons structure/organisation ont été envisagées.

4.1 Structure 1 : nabla/delta distincts et quatre vecteurs X/Y

La première option conserve ∇ et Δ dans deux tableaux séparés, et utilise quatre vecteurs distincts X_{old}, X_{new}, Y_{old}, Y_{new} pour l'itération :

```
struct Matrice {
    int n;
    long long n_links;
    double* values;
    long long* col_ptr;
    int* row_idx;
    int* degree_Sort_Tot;
    double* nabla;   // nabla[j] = min colonne j de G
    double* delta;   // delta[j] = max colonne j de G
};

// Vecteurs d'itération
double* X_old = new double[N];
double* X_new = new double[N];
double* Y_old = new double[N];
double* Y_new = new double[N];
```

Cette organisation est naturelle et lisible. Cependant, à chaque itération le processeur sollicite six zones mémoire distinctes (∇, Δ, X_{old}, Y_{old}, X_{new}, Y_{new}), ce qui augmente la pression sur le cache pour les grands graphes. En particulier, les accès à $\nabla[j]$ et $\Delta[j]$ dans la boucle interne proviennent de deux zones distantes, risquant deux *cache misses* distincts à chaque itération de colonne.

4.2 Structure 2 : nabla/delta entrelacés et deux vecteurs X/Y entrelacés

La seconde option entrelacer ∇ et Δ dans un seul tableau `nd`, et fusionne X et Y dans deux vecteurs entrelacés Z_{old} et Z_{new} :

```
struct Matrice {
    int n;
    long long n_links;
    double* values;
    long long* col_ptr;
    int* row_idx;
    int* degree_Sort_Tot;
    double* nd;   // nd[2j] = nabla[j],  nd[2j+1] = delta[j]
};

// Vecteurs d'itération entrelacés
double* Z_old = new double[2*N]; // Z_old[2i] = X[i], Z_old[2i+1] = Y[i]
double* Z_new = new double[2*N];
```

Avec cette organisation, $\nabla[j]$ et $\Delta[j]$ sont contigus en mémoire, tout comme X_i et Y_i. Lorsque le processeur charge $\nabla[j]$ pour `max(xg_j, nabla[j])`, il charge automatiquement $\Delta[j]$ dans la même ligne de cache. De même, lors du produit $X \cdot G$, le chargement de X_i entraîne automatiquement celui de Y_i dans la même ligne de cache. Ces deux effets combinés réduisent significativement le nombre de *cache misses* sur les graphes de grande taille.

Ces deux structures seront comparées expérimentalement dans la section des résultats.

5 Environnement expérimental

Les expériences ont été conduites sur la machine suivante :

- **Processeur** : AMD Ryzen 5 4600H (6 cœurs, 12 threads, jusqu'à 4.0 GHz en mode Turbo)
- **Mémoire vive** : 23,42 Go
- **Système d'exploitation** : Microsoft Windows 11 Famille
- **Compilateur** : g++ avec les options `-O2 -fopenmp`

Les trois matrices de test sont issues de la collection SuiteSparse Matrix Collection :

Matrice	**Nœuds** (N)	**Liens** (nz)
Harvard500	500	2,636
web-Google	875,713	5,105,039
uk-2002	18,520,486	298,113,762

La grille expérimentale couvre les combinaisons suivantes :

- $\alpha \in \{0.1,\ 0.15,\ \ldots,\ 0.9\}$ (17 valeurs)
- $\varepsilon \in \{10^{-6},\ 10^{-7},\ \ldots,\ 10^{-12}\}$ (7 valeurs)
- $t_{\max} \in \{1,\ 2,\ 5,\ 10,\ 25,\ 50,\ 100\}$ secondes (7 valeurs)

Soit $17 \times 7 \times 7 = 833$ configurations par matrice, chacune exécutée pour les trois méthodes, pour un total de $833 \times 3 = 2{,}499$ exécutions du PageRank par matrice. Chaque exécution s'arrête dès que l'une des conditions suivantes est atteinte : convergence ($\|\cdot\|_1 < \varepsilon$), dépassement du temps imparti ($t_{\max}$), ou atteinte du nombre maximal d'itérations (1000).

6 Méthodologie d'évaluation

6.1 Démarche générale

L'évaluation expérimentale repose sur une comparaison systématique des trois méthodes pour chaque triplet $(\alpha, \varepsilon, t_{\max})$. Pour chaque configuration, les trois algorithmes sont exécutés séquentiellement sur la même matrice déjà chargée en mémoire, afin d'éliminer tout biais lié au temps de chargement ou à la contention entre processus. Les mesures de temps sont effectuées avec `std::chrono::steady_clock` directement dans le code C++.

6.2 Raison d'arrêt

Chaque exécution enregistre la raison d'arrêt parmi : convergence (critère ε atteint), timeout ($t_{\max}$ dépassé), ou nombre maximal d'itérations atteint. Cette information est cruciale pour distinguer les configurations où les méthodes ont réellement convergé de celles où elles ont été interrompues.

6.3 Mesure de la qualité

Les résultats sont enregistrés dans deux fichiers CSV :

- `results.csv` : une ligne par configuration, contenant pour chaque méthode le nombre d'itérations, le temps d'exécution et la raison d'arrêt.
- `classements.csv` : le Top 20 des pages pour chaque exécution, avec leur score PageRank.

Ces deux fichiers sont joinables via un identifiant unique (timestamp de démarrage) pour corréler performances et qualité des résultats.

6.4 Référence de qualité

Pour mesurer la qualité des scores obtenus, une référence est construite pour chaque couple (α, ε) : il s'agit du classement produit par la méthode classique avec $t_{\max} = 100$s et $\varepsilon = 10^{-12}$, configuration pour laquelle aucun timeout n'est observé. L'erreur de qualité est ensuite définie comme l'erreur L_1 sur les scores du Top 20 de référence :

$$\text{Erreur} = \sum_{p \in \text{Top}_{20}^{\text{ref}}} |s_{\text{ref}}(p) - s_{\text{méthode}}(p)|$$

où $s_{\text{méthode}}(p) = 0$ si la page p n'apparaît pas dans le Top 20 de la méthode testée. Cette métrique page par page garantit que l'on compare bien les mêmes nœuds entre méthodes.

6.5 Reproductibilité

Afin d'assurer la reproductibilité des mesures temporelles, les expériences sont lancées via un script PowerShell qui active le mode haute performance du système, désactive la mise en veille, et configure le processeur en mode Turbo. Les expériences sur les différentes matrices sont exécutées séquentiellement pour éviter toute contention mémoire ou CPU entre processus concurrents.

7 Résultats et analyse

7.1 Influence du facteur d'amortissement α

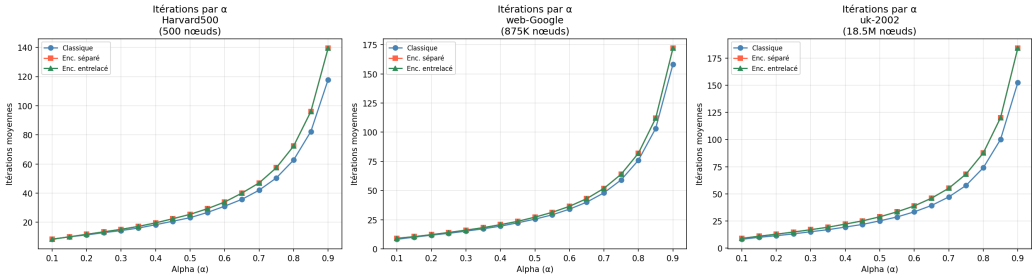


Figure 1: Nombre moyen d'itérations en fonction de α pour les trois matrices ($t_{\max} = 100$s).

La figure 1 montre que le nombre d'itérations croît fortement avec α pour les trois méthodes et les trois matrices. Ce comportement s'explique par le fait qu'un α élevé amplifie le poids des liens du graphe au détriment de la téléportation uniforme, rendant la matrice G moins contractante et ralentissant la convergence. Le second vecteur propre de G, qui gouverne la vitesse de convergence, se rapproche de 1 lorsque $\alpha \to 1$.

Pour $\alpha \leq 0.5$, les trois méthodes convergent en un nombre d'itérations similaire et relativement faible. Pour $\alpha \geq 0.7$, les différences entre méthodes deviennent visibles : l'encadrement requiert en moyenne 16% d'itérations supplémentaires par rapport au classique sur uk-2002. Ce surcoût s'explique par la différence de critère de convergence : la méthode classique mesure $\|x^{(k+1)} - x^{(k)}\|_1$, tandis que l'encadrement mesure $\|Y^{(k)} - X^{(k)}\|_1$, une quantité intrinsèquement plus conservative.

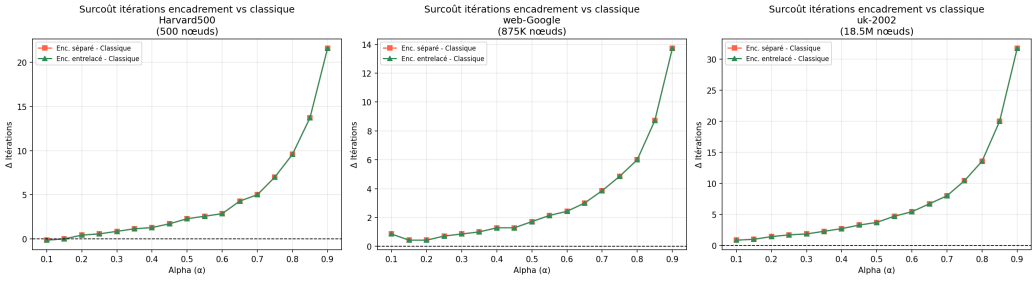


Figure 2: Surcoût en itérations des méthodes d'encadrement par rapport au classique.

La figure 2 confirme ce surcoût en affichant directement la différence $\text{Iter}_{\text{encadrement}} - \text{Iter}_{\text{classique}}$. Sur Harvard500, le surcoût est quasi nul pour les petits α mais atteint 15 à 20 itérations supplémentaires pour $\alpha = 0.9$. Sur uk-2002, le surcoût est plus régulier et croît de façon quasi linéaire avec α. Il est important de noter que les deux variantes d'encadrement (séparée et entrelacée) produisent exactement le même nombre d'itérations : l'entrelacement est une optimisation purement mémoire qui n'affecte pas la trajectoire algorithmique.

Intervalle pertinent pour α : les valeurs $\alpha \in [0.5, 0.9]$ sont les plus discriminantes pour comparer les méthodes. En dessous de 0.5, la convergence est trop rapide pour observer des différences significatives ; au-dessus de 0.9, les temps de calcul deviennent prohibitifs pour les grands graphes. La valeur standard $\alpha = 0.85$ se situe dans la zone la plus intéressante.

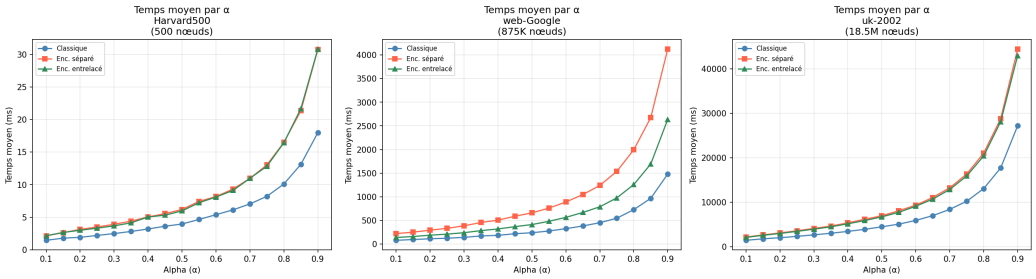


Figure 3: Temps moyen d'exécution en fonction de α ($t_{\max} = 100$s).

La figure 3 illustre l'impact de α sur les temps d'exécution. L'encadrement est systématiquement plus lent que le classique, car il effectue deux produits vecteur-matrice par itération ($X \cdot G$ et $Y \cdot G$) au lieu d'un seul. Sur uk-2002, l'encadrement séparé est en moyenne 1,58 fois plus lent que le classique. Sur web-Google, ce ratio monte à 2,73 fois, mais l'entrelacement le ramène à 1,73 grâce au gain de localité cache.

7.2 Impact de l'entrelacement mémoire

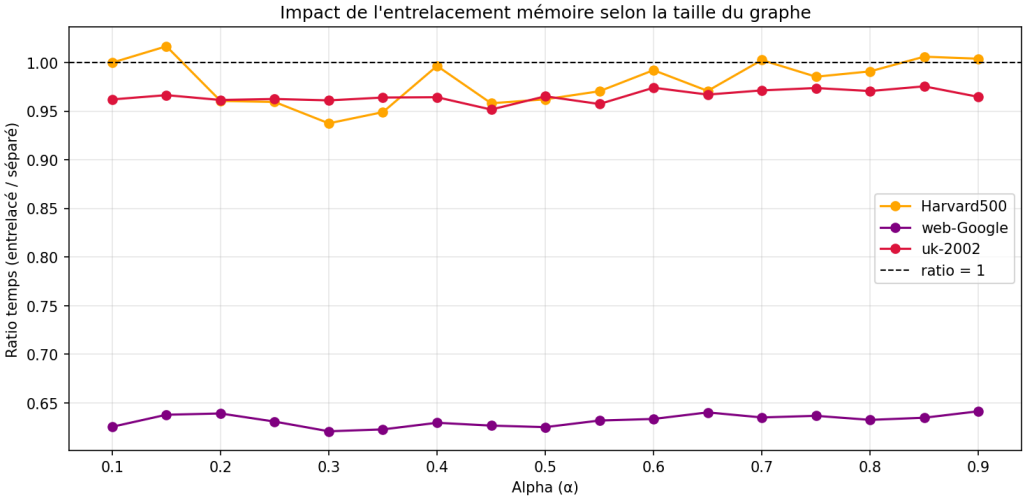


Figure 4: Ratio du temps d'exécution entre encadrement entrelacé et encadrement séparé.

La figure 4 présente le ratio des temps d'exécution entre les deux variantes de l'encadrement. Un ratio inférieur à 1 indique que l'entrelacement est plus rapide que les vecteurs séparés.

Sur **Harvard500**, le ratio est proche de 1 quelle que soit la valeur de α : le graphe est trop petit (500 nœuds) pour que la localité cache joue un rôle, car l'intégralité des vecteurs tient dans les niveaux de cache du processeur.

Sur **web-Google**, l'entrelacement procure un gain allant jusqu'à 37% selon α. Ce gain s'explique par la taille intermédiaire du graphe (875K nœuds) : les vecteurs X et Y ne tiennent plus dans le cache L2, et le fait de les charger conjointement en mémoire réduit significativement le nombre de *cache misses* lors des accès X_i et Y_i dans la boucle interne.

Sur **uk-2002**, le gain est plus modeste (environ 3%). Malgré la très grande taille du graphe, les accès mémoire sont dominés par les accès irréguliers aux indices `row_idx` de la matrice CSC, qui génèrent des *cache misses* quel que soit le layout des vecteurs. L'entrelacement reste bénéfique mais son impact est relativisé par ce bruit structurel.

Conclusion : l'entrelacement mémoire est une optimisation conditionnelle à la taille du graphe. Il est neutre sur les petits graphes, bénéfique sur les graphes intermédiaires, et partiellement masqué par les accès irréguliers sur les très grands graphes.

7.3 Influence du critère de convergence ε

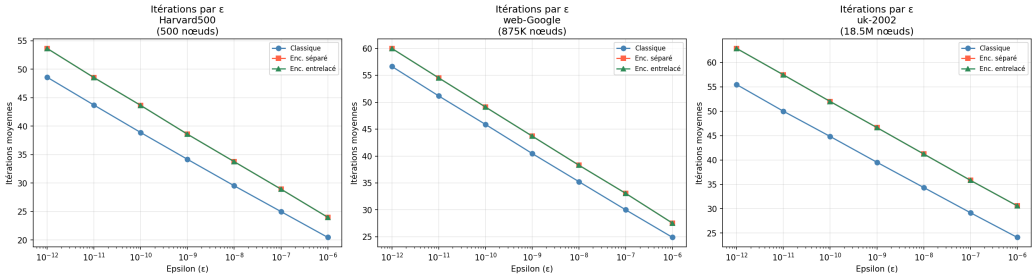


Figure 5: Nombre moyen d'itérations en fonction de ε ($t_{\max} = 100$s).

La figure 5 montre que le nombre d'itérations croît de manière quasi linéaire en échelle logarithmique avec la précision demandée, ce qui est cohérent avec la convergence géométrique de la méthode des puissances. Chaque décade supplémentaire de précision ($\varepsilon/10$) coûte environ 3 à 5 itérations supplémentaires selon la matrice et la méthode.

Les trois méthodes présentent des pentes similaires, ce qui indique que la vitesse de convergence intrinsèque (liée au second vecteur propre de G) est la même pour toutes. L'encadrement requiert simplement un point de départ plus élevé car son critère $\|Y - X\|_1 < \varepsilon$ est plus strict.

Intervalle pertinent pour ε : les valeurs $\varepsilon \in [10^{-6},\ 10^{-9}]$ constituent la plage la plus pertinente en pratique. En dessous de 10^{-9}, les différences de classement entre les méthodes disparaissent et le surcoût de calcul n'apporte pas de gain observable. Au-delà de 10^{-6}, les résultats peuvent être trop imprécis pour des applications réelles.

7.4 Qualité des résultats en fonction de ε

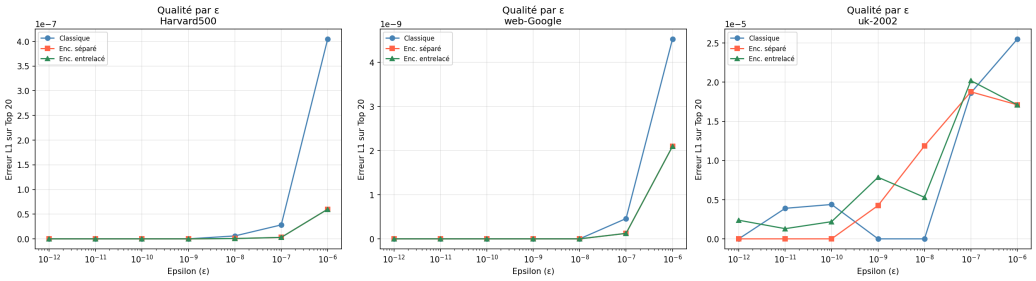


Figure 6: Erreur L_1 sur le Top 20 en fonction de ε (référence : classique, $\varepsilon = 10^{-12}$, $t_{\max} = 100$s).

La figure 6 compare la qualité des scores du Top 20 pour les configurations sans timeout. L'erreur est mesurée page par page par rapport à une référence convergée.

Sur **Harvard500**, le classique produit des scores légèrement plus proches de la référence que l'encadrement pour les grands ε. Cela s'explique par le fait que sur un petit graphe, le critère $\|x^{(k+1)} - x^{(k)}\|_1 < \varepsilon$ et le critère $\|Y - X\|_1 < \varepsilon$ ne se traduisent pas par la même précision effective.

Sur **web-Google**, les deux variantes d'encadrement produisent des scores plus précis que le classique pour $\varepsilon \in [10^{-6}, 10^{-8}]$. Ce résultat s'explique par la nature du critère d'arrêt de l'encadrement : $\|Y - X\|_1 < \varepsilon$ borne directement l'amplitude de l'intervalle d'incertitude sur x, tandis que $\|x^{(k+1)} - x^{(k)}\|_1 < \varepsilon$ ne garantit pas que l'itérée est proche de la solution exacte. L'encadrement est donc intrinsèquement plus informatif.

Sur **uk-2002**, les erreurs sont du même ordre de grandeur pour les trois méthodes, sans avantage clair de l'encadrement.

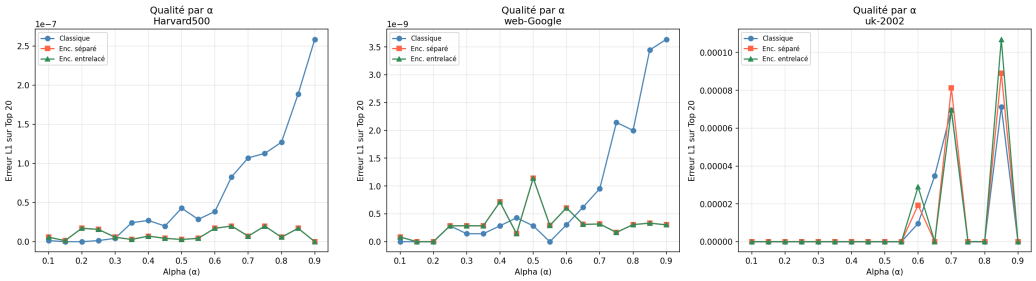


Figure 7: Erreur L_1 sur le Top 20 en fonction de α (référence : classique, $\varepsilon = 10^{-12}$, $t_{\max} = 100$s).

La figure 7 montre que la qualité des scores dépend fortement de α : pour les grands α, les scores des pages importantes sont plus étalés, rendant les erreurs plus visibles. Pour les petits α, les scores sont plus uniformes et les erreurs plus faibles. Cette observation confirme que α est le paramètre le plus influent sur la fois la vitesse de convergence et la qualité des résultats intermédiaires.

7.5 Accord sur le Top 20

Les trois méthodes produisent des classements très cohérents entre elles. Sur Harvard500, le Top 20 est identique à 100% entre les trois méthodes pour toutes les configurations testées. Sur web-Google, l'overlap moyen est supérieur à 99%. Sur uk-2002, il atteint 96 à 98% selon les paramètres,

les rares divergences concernant les dernières positions du classement où les scores sont très proches et peuvent s'intervertir sans impact significatif.

8 Structure retenue

À l'issue des expériences, la structure retenue pour l'implémentation finale de l'encadrement est celle avec les vecteurs entrelacés :

```
struct Matrice {
    int n;
    long long n_links;
    double* values;
    long long* col_ptr;
    int* row_idx;
    int* degree_Sort_Tot;
    double* nd;   // nd[2j] = nabla[j],  nd[2j+1] = delta[j]
};
```

et les vecteurs X et Y organisés en deux tableaux entrelacés Z_{old} et Z_{new} avec $Z[2i] = X_i$ et $Z[2i+1] = Y_i$.

8.1 Justification

Les résultats expérimentaux (figure 4) montrent que l'entrelacement procure un gain allant jusqu'à 37% sur web-Google par rapport à la variante quatre vecteurs distincts. Sur uk-2002, le gain est plus modeste (environ 3%) mais reste positif. Sur Harvard500, les deux organisations sont équivalentes car le graphe est trop petit pour que la localité cache joue un rôle.

L'entrelacement est donc la structure dominante ou égale dans tous les cas testés. Elle ne présente aucun inconvénient algorithmique — le nombre d'itérations est rigoureusement identique entre les deux variantes — et offre un avantage mémoire sur les graphes de grande taille. C'est donc la structure retenue pour l'implémentation finale.

9 Difficultés rencontrées et perspectives

9.1 Invariant d'encadrement et normalisation séparée

La principale difficulté théorique rencontrée concerne le maintien de l'invariant $X^{(k)}[i] \leq x[i] \leq Y^{(k)}[i]$ à chaque itération. L'algorithme tel que donné dans l'énoncé normalise X et Y séparément, en divisant chacun par sa propre norme L_1. Or, nous avons observé expérimentalement que $\|\nabla\|_1 \approx 0.15$ tandis que $\|\Delta\|_1 \approx 72.9$ sur uk-2002, soit un rapport de l'ordre de 486. Cette asymétrie fait que dès l'initialisation, $X^{(0)}[i] = \nabla[i]/\|\nabla\|_1$ peut être supérieur à $Y^{(0)}[i] = \Delta[i]/\|\Delta\|_1$ pour de nombreuses composantes, ce qui viole l'invariant avant même la première itération.

Nous avons tenté plusieurs corrections :

- **Normalisation par une norme commune** $(\|\nabla\|_1+\|\Delta\|_1)/2$: garantit $X^{(0)} \leq Y^{(0)}$ à l'initialisation, mais la normalisation séparée aux itérations suivantes réintroduit des inversions.
- **Initialisation uniforme** $X^{(0)}[i] = Y^{(0)}[i] = 1/N$: garantit l'absence d'inversions au départ, mais ne respecte plus l'initialisation prescrite par l'algorithme.
- **Correction par moyenne** après chaque itération : remplacer les composantes inversées par $(X[i] + Y[i])/2$. Cette approche restaure localement l'ordre, mais perturbe les normes L_1 et nécessite une renormalisation qui peut réintroduire des inversions.
- **Inversion des normes** à l'initialisation, en normalisant ∇ par $\|\Delta\|_1$ et Δ par $\|\nabla\|_1$: supprime les inversions initiales mais produit un intervalle $[X, Y]$ trop large, ce qui empêche la convergence.

Aucune de ces approches n'a permis de maintenir strictement l'invariant à chaque itération tout en conservant la convergence de l'algorithme. Nous avons conclu que la normalisation séparée telle que prescrite dans l'énoncé est incompatible avec le maintien strict de l'invariant composante par composante, sauf à convergence.

9.2 Conséquences et ce qu'on aurait pu obtenir

Si l'invariant $X^{(k)}[i] \leq x[i] \leq Y^{(k)}[i]$ avait pu être maintenu à chaque itération, les bénéfices auraient été considérables :

- **Borne certifiée sur l'erreur à tout moment** : la quantité $\|Y^{(k)} - X^{(k)}\|_1$ aurait constitué une borne garantie sur l'erreur totale d'approximation, pas seulement un indicateur de convergence. L'algorithme aurait pu s'arrêter dès que la précision souhaitée est prouvée atteinte.

- **Score garanti** : le score $(X_i + Y_i)/2$ aurait été garanti à distance au plus $(Y_i - X_i)/2$ de la vraie valeur $x[i]$, fournissant une borne d'erreur par composante.

- **Critère d'arrêt plus fort** : $\|Y^{(k)} - X^{(k)}\|_1 < \varepsilon$ aurait signifié que l'erreur sur chaque page est bornée par ε, une garantie bien plus forte que celle offerte par le critère classique $\|x^{(k+1)} - x^{(k)}\|_1 < \varepsilon$, qui ne borne que la variation entre deux itérées consécutives sans garantir la proximité à la solution exacte.

- **Meilleure qualité sous contrainte de temps** : avec un encadrement strict, les scores intermédiaires obtenus sous timeout auraient été plus proches de la distribution stationnaire, car chaque itération aurait serré l'intervalle de façon monotone et garantie.

En pratique, nous observons que les inversions à convergence sont de l'ordre de 5×10^{-9}, ce qui est négligeable. L'algorithme converge correctement et produit des classements cohérents avec la méthode classique. L'invariant est donc vérifié *asymptotiquement*, à la précision machine près, même s'il n'est pas maintenu strictement aux itérations intermédiaires.

9.3 Autres difficultés

Un second bug a été identifié en cours d'expérimentation dans le module d'extraction du Top 20 pour la variante entrelacée : l'indice mémoire $j = 2i$ était stocké à la place de l'indice réel du nœud i, produisant des classements avec des identifiants de pages doublés. Ce bug a été corrigé avant les expériences finales.

Par ailleurs, le calcul initial des bornes ∇ et Δ utilisait $\nabla[j] = \text{sec_term}$ pour toutes les colonnes, ce qui correspondait au minimum global de G plutôt qu'au minimum réel de chaque colonne. Cette approximation pessimiste élargissait artificiellement l'intervalle initial et ralentissait la convergence. La correction, qui calcule le vrai minimum de chaque colonne en O(NZ), a permis de réduire le nombre d'itérations de l'encadrement et d'améliorer la qualité des scores intermédiaires.

10 Conclusion

Ce travail a permis de comparer expérimentalement trois implémentations du PageRank sur des graphes web de tailles très différentes, en faisant varier systématiquement les paramètres α et ε.

L'algorithme d'encadrement converge vers la même distribution stationnaire que la méthode classique, avec un surcoût en itérations de l'ordre de 7 à 16% selon la taille du graphe et la valeur de α. Ce surcoût est la contrepartie d'un critère d'arrêt plus informatif : $\|Y - X\|_1 < \varepsilon$ fournit une borne certifiée sur l'erreur d'approximation, contrairement au critère classique. Cet avantage se manifeste concrètement sur web-Google où l'encadrement produit des scores plus précis que le classique pour un même ε.

L'entrelacement mémoire des vecteurs X et Y apporte un gain significatif sur les graphes de taille intermédiaire (jusqu'à 37% sur web-Google), mais est neutre sur les petits graphes et partiellement masqué par les accès irréguliers sur les très grands graphes. Cette optimisation est donc conditionnelle à la taille du problème.

Enfin, les paramètres les plus pertinents à étudier sont $\alpha \in [0.5, 0.9]$ et $\varepsilon \in [10^{-6}, 10^{-9}]$: en dehors de ces plages, les différences entre méthodes s'atténuent et l'analyse comparative perd de son intérêt. La valeur standard $\alpha = 0.85$ constitue le cas de référence le plus représentatif.